

# Sistema de Finanzas Personales

## Proyecto 2 — Paradigma Funcional

|                  |                                                      |
|------------------|------------------------------------------------------|
| Curso            | Lenguajes de Programación                            |
| Semestre         | I Semestre 2026                                      |
| Integrantes      | Justin Lacayo, Frederick Fernández,<br>Josimar Araya |
| Fecha de entrega | 11 de mayo de 2026                                   |
| Tecnología       | Haskell (GHC)                                        |

# Índice

---

1. Enlace al repositorio
2. Pasos de instalación
  - 2.1 Instalar GHC
  - 2.2 Dependencias
  - 2.3 Compilación
  - 2.4 Ejecución
3. Manual de usuario
  - 3.1 Inicio del programa
  - 3.2 Menú principal
  - 3.3 Agregar registro
  - 3.4 Ver registros con filtros
  - 3.5 Gestionar presupuestos
  - 3.6 Gestionar reglas
  - 3.7 Análisis financiero
  - 3.8 Simulación de escenarios
  - 3.9 Generar reportes
  - 3.10 Salir
4. Arquitectura lógica
  - 4.1 Visión general
  - 4.2 Descripción de cada módulo
  - 4.3 Diagrama de dependencias
  - 4.4 Flujo de datos

# 1. Enlace al repositorio

---

El código fuente del proyecto se encuentra en:

<https://github.com/ItsJustStyles/Lenguajes-de-programacion>

El código del Proyecto 2 se ubica dentro de la carpeta **Proyecto 2/** en la rama **main**.

---

## 2. Pasos de instalación

---

### 2.1 Instalar GHC

El proyecto requiere **GHC 9.x** (Glasgow Haskell Compiler). Se recomienda instalarlo a través de **GHCup**, el instalador oficial de la toolchain de Haskell.

**Linux / macOS / WSL:**

```
| curl --proto '=https' --tlsv1.2 -sSf https://get-ghcup.haskell.org | sh
```

Durante la instalación, aceptar la instalación de GHC y agregar GHCup al PATH. Luego reiniciar la terminal o ejecutar:

```
| source ~/.ghcup/env
```

Verificar la instalación:

```
| ghc --version  
# GHC version 9.x.x
```

**Windows (nativo):**

Descargar el instalador desde la página oficial de GHCup: <https://www.haskell.org/ghcup/>

### 2.2 Dependencias

El proyecto utiliza únicamente librerías incluidas en la distribución estándar de GHC (**base** y **time**). No se requiere instalar paquetes adicionales con Cabal ni Stack.

| Librería   | Módulos usados                                                                                       |
|------------|------------------------------------------------------------------------------------------------------|
| base       | Data.Char, Data.List, Data.Ord, System.IO, System.Directory, System.FilePath, Text.Read, Text.Printf |
| time       | Data.Time.Clock, Data.Time.Calendar, Data.Time.Format                                                |
| containers | Data.Map.Strict                                                                                      |

*Nota: El módulo containers generalmente viene preinstalado con GHC. Si no está disponible, instalarlo con: cabal install containers*

### 2.3 Compilación

1. Clonar el repositorio:

```
| git clone https://github.com/ItsJustStyles/Lenguajes-de-programacion.git
```

```
| cd "Lenguajes-de-programacion/Proyecto 2"
```

## 2. Compilar todos los módulos con GHC:

```
| ghc -o Main Main.hs Logica.hs Interfaz.hs Persistencia.hs Tipos.hs  
Validaciones.hs
```

GHC compilará automáticamente todos los módulos en el orden correcto. Se generarán los archivos .hi (interfaces) y .o (objetos) junto con el ejecutable Main.

## 2.4 Ejecución

```
| ./Main
```

### En Windows:

```
| Main.exe
```

Al iniciarse, el programa intentará cargar los archivos de datos desde la carpeta Reportes/. Si no existen (primera ejecución), comenzará con listas vacías.

---

### 3. Manual de usuario

#### 3.1 Inicio del programa

Al ejecutar el programa se muestra el encabezado de bienvenida con la cantidad de registros, presupuestos y reglas cargados:

```
-----  
Sistema de Finanzas Personales  
-----  
Registros cargados:      3  
Presupuestos cargados:  1  
Reglas cargadas:         2
```

#### 3.2 Menú principal

El menú principal presenta 8 opciones numeradas. Se navega digitando el número de la opción y presionando Enter.

```
|| SISTEMA DE FINANZAS PERSONALES ||  
||  
|| 1. Agregar registro ||  
|| 2. Ver registros con filtros ||  
|| 3. Gestionar presupuestos ||  
|| 4. Gestionar reglas ||  
|| 5. Ver analisis financiero ||  
|| 6. Simulacion de escenarios ||  
|| 7. Generar reportes ||  
|| 8. Salir ||  
||  
||  
Opcion:
```

Si se digita una opción inválida, el menú se vuelve a mostrar.

#### 3.3 Agregar registro (Opción 1)

Permite registrar una nueva transacción financiera. El sistema solicita:

| Campo     | Descripción             | Formato / Ejemplo                         |
|-----------|-------------------------|-------------------------------------------|
| Tipo      | Tipo de transacción     | 1=Ingreso, 2=Gasto, 3=Ahorro, 4=Inversion |
| Monto     | Cantidad en colones     | Número decimal positivo, ej: 25000.50     |
| Categoría | Categoría del registro  | Texto libre, ej: Alimentación             |
| Fecha     | Fecha de la transacción | Formato YYYY-MM-DD, ej: 2026-05-10        |

|             |                      |                                      |
|-------------|----------------------|--------------------------------------|
| Descripción | Detalle del registro | Texto libre, ej: Compra supermercado |
| Etiquetas   | Etiquetas opcionales | Separadas por coma, ej: fijo,mensual |

#### El sistema valida cada campo antes de guardar:

- El monto debe ser positivo.
- La fecha debe tener el formato YYYY-MM-DD.
- La categoría y descripción no pueden estar vacías.
- Se verifica coherencia entre tipo y categoría.

Si la validación falla, se muestra el error y el registro no se agrega.

### 3.4 Ver registros con filtros (Opción 2)

Permite consultar los registros existentes con 5 tipos de filtro:

| Sub-opción             | Filtro                                              |
|------------------------|-----------------------------------------------------|
| 1. Ver todos           | Muestra todos los registros sin filtro              |
| 2. Por tipo            | Filtra por Ingreso, Gasto, Ahorro o Inversion       |
| 3. Por categoría       | Busca registros de una categoría específica         |
| 4. Por etiqueta        | Muestra registros que contienen cierta etiqueta     |
| 5. Por rango de fechas | Solicita fecha inicial y final (formato YYYY-MM-DD) |
| 6. Volver              | Regresa al menú principal                           |

Cada registro se muestra con todos sus campos:

```
#1
Tipo:      Ingreso
Monto:     $50000.00
Categoria: Salario
Fecha:     2026-05-09
Descripcion: Pago
Etiquetas: fijo
```

### 3.5 Gestionar presupuestos (Opción 3)

Un presupuesto define un límite de gasto mensual por categoría. Las sub-opciones son:

| Sub-opción          | Acción                                     |
|---------------------|--------------------------------------------|
| 1. Ver presupuestos | Lista todos los presupuestos con su límite |

|                         |                                                                       |
|-------------------------|-----------------------------------------------------------------------|
| 2. Agregar o actualizar | Crea un nuevo presupuesto o actualiza el existente para esa categoría |
| 3. Eliminar             | Elimina un presupuesto seleccionado por número                        |
| 4. Verificar alertas    | Compara gastos reales contra límites y muestra alertas                |
| 5. Volver               | Regresa al menú principal                                             |

Ejemplo de alerta:

```
- ALERTA: La categoria 'Alimentación' supero el presupuesto. Gasto real:
35000.0 | Limite: 20000.0
```

### 3.6 Gestionar reglas (Opción 4)

Las reglas son condiciones de alerta automatizadas. Existen dos tipos:

- ReglaGastoMax: alerta si los gastos en una categoría superan un monto definido.
- ReglasAhorroMin: alerta si el ahorro acumulado total es inferior a un mínimo.

| Sub-opción                | Acción                                            |
|---------------------------|---------------------------------------------------|
| 1. Ver reglas             | Lista todas las reglas activas                    |
| 2. Regla de gasto máximo  | Crea una regla por categoría con un tope de gasto |
| 3. Regla de ahorro mínimo | Crea una regla de ahorro mínimo acumulado         |
| 4. Eliminar regla         | Elimina una regla por número                      |
| 5. Evaluar reglas         | Evalúa todas las reglas y muestra alertas         |
| 6. Volver                 | Regresa al menú principal                         |

### 3.7 Análisis financiero (Opción 5)

Muestra un análisis completo sin necesidad de configuración adicional. Incluye:

- Resumen general: totales de ingresos, gastos, ahorros e inversiones, balance neto y proyección de gasto mensual promedio.
- Flujo de caja mensual: balance (ingresos – gastos) por cada mes con datos.
- Tendencia de gastos: evolución mensual con indicación de aumento o disminución respecto al mes anterior.
- Categorías con mayor gasto: top 5 categorías ordenadas por monto gastado.
- Alertas activas: todas las alertas de presupuestos y reglas vigentes.

Si no hay registros, el sistema lo informa y no realiza cálculos.

### 3.8 Simulación de escenarios (Opción 6)

Permite proyectar el impacto financiero de reducir los gastos. El sistema solicita:

3. Porcentaje de reducción de gastos (ej: 20 para 20%).
4. Cantidad de meses a proyectar.

| Dato                        | Descripción                                           |
|-----------------------------|-------------------------------------------------------|
| Gasto actual total          | Suma de todos los gastos registrados                  |
| Gasto simulado total        | Gasto aplicando la reducción                          |
| Ahorro por reducción        | Diferencia entre gasto actual y simulado              |
| Balance actual              | Ingresos – Gastos actuales                            |
| Balance con escenario       | Ingresos – Gastos simulados                           |
| Ahorro histórico proyectado | Proyección basada en el promedio histórico de ahorros |
| Ahorro extra en N meses     | Ahorro adicional estimado para el período indicado    |

### 3.9 Generar reportes (Opción 7)

Genera reportes en texto plano y los guarda en la carpeta Reportes/:

| Sub-opción                    | Archivo generado                            |
|-------------------------------|---------------------------------------------|
| 1. Reporte mensual            | Reportes/reporte_mensual_YYYY_MM.txt        |
| 2. Comparación de periodos    | Reportes/reporte_comparacion_periodos.txt   |
| 3. Categorías con mayor gasto | Reportes/reporte_categorias_mayor_gasto.txt |

Para el reporte mensual se solicita año y mes (1–12). Para la comparación de periodos se piden las fechas de inicio y fin de dos periodos distintos. Para categorías se pide la cantidad de categorías a incluir.

### 3.10 Salir (Opción 8)

Al seleccionar esta opción, el programa guarda automáticamente todos los datos:

| Archivo                   | Contenido                  |
|---------------------------|----------------------------|
| Reportes/finanzas.csv     | Registros financieros      |
| Reportes/presupuestos.csv | Presupuestos por categoría |
| Reportes/reglas.csv       | Reglas de alerta           |

Luego muestra la confirmación:

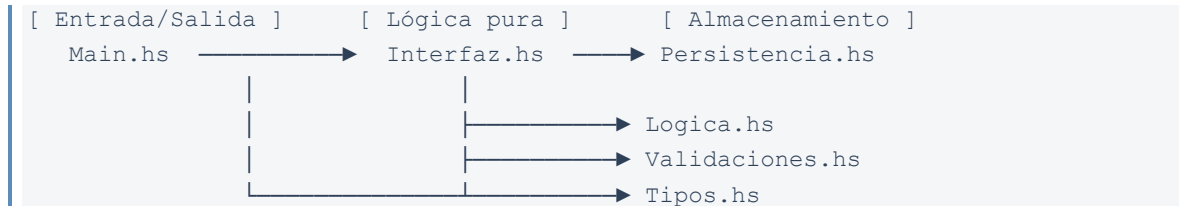
```
[OK] Datos guardados correctamente :D
El programa ha finalizado
```



## 4. Arquitectura lógica

### 4.1 Visión general

El sistema está organizado en **6 módulos de Haskell**, cada uno con una responsabilidad única y bien delimitada. La arquitectura sigue el principio de separación de capas:



### 4.2 Descripción de cada módulo

#### Tipos.hs — Definiciones de tipos de datos

Es la base de todo el sistema. Define los tipos algebraicos que usan todos los demás módulos. No importa ningún otro módulo del proyecto.

**Tipos definidos:**

- TipoRegistro — suma algebraica: Ingreso | Gasto | Ahorro | Inversion
- RegistroFinanciero — registro con campos: monto, categoría, fecha, descripción, etiquetas y tipo
- Presupuesto — par (categoría, límite de gasto)
- Regla — suma algebraica: ReglaGastoMax Categoria Monto | ReglasAhorroMin Monto

Los tipos derivan Show, Read y Eq para facilitar la serialización y comparación.

#### Validaciones.hs — Validación de entradas

Contiene funciones puras que validan los datos ingresados por el usuario. Usa el tipo **Either String a** para representar éxito (Right) o error (Left).

**Funciones principales:**

- validarMonto — rechaza montos no positivos
- validarFecha — parsea el formato YYYY-MM-DD
- validarCategoria / validarDescripcion — rechazan cadenas vacías
- validarCoherenciaTipo — detecta incoherencias entre tipo y categoría
- validarRegistro — combina todas las validaciones usando la mónada Either

#### Logica.hs — Lógica de negocio

Contiene todo el procesamiento de datos financieros. Son funciones completamente puras (sin efectos de I/O), lo que permite probarlas de forma aislada y componerlas libremente.

| Grupo    | Funciones clave                                                               |
|----------|-------------------------------------------------------------------------------|
| Filtrado | filtrarPorTipo, filtrarPorCategoria, filtrarPorRangoFecha, filtrarPorEtiqueta |

|                   |                                                                         |
|-------------------|-------------------------------------------------------------------------|
| Cálculos básicos  | totalMonto, totalPorTipo, balance                                       |
| Agrupación        | agruparPorCategoria (con Data.Map), agruparPorMes                       |
| Análisis avanzado | flujoCajaMensual, tendenciaGasto, proyeccionGasto, categoriasMayorGasto |
| Simulación        | simularReduccionGastos, proyeccionAhorro                                |
| Presupuestos      | gastoRealPorCategoria, verificarPresupuesto, verificarPresupuestos      |
| Reglas            | evaluarRegla, evaluarReglas                                             |
| Reportes          | resumenMensual, comparacionPeriodos, categoriasMayorGasto               |

### Persistencia.hs — Almacenamiento en CSV

Gestiona la lectura y escritura de datos en archivos CSV usando | como separador de campos y ; para separar etiquetas dentro de un campo.

#### Responsabilidades:

- Define las rutas: Reportes/finanzas.csv, Reportes/presupuestos.csv, Reportes/reglas.csv
- Serializa y deserializa cada tipo de dato
- Maneja la existencia o ausencia de archivos con doesFileExist
- Crea directorios automáticamente con createDirectoryIfMissing
- Expone una API pública de 6 funciones: guardarDatos, cargarDatos, guardarPresupuestos, cargarPresupuestos, guardarReglas, cargarReglas

### Interfaz.hs — Interfaz de usuario en terminal

Contiene toda la lógica de presentación e interacción con el usuario.

- Renderizado de menús con marcos de caracteres Unicode (┌, ┐, └, ┘, etc.)
- Lectura y validación de entradas con leerDouble, leerInt, leerFechaValidada
- Implementa los 7 submenús del sistema
- Formatea montos con símbolo ₡ y dos decimales
- El estado del sistema se pasa como parámetros siguiendo el estilo funcional puro con recursión de cola

### Main.hs — Punto de entrada

Orquesta el flujo completo del programa:

5. Deshabilita el buffer de stdout para respuesta inmediata
6. Carga los tres conjuntos de datos desde disco (cargarDatos, cargarPresupuestos, cargarReglas)
7. Muestra estadísticas de datos cargados
8. Delega el control a iniciarInterfaz
9. Al retornar, guarda los datos finales en disco

## 4.3 Diagrama de dependencias

```

Main.hs
├─ importa Interfaz      (iniciarInterfaz)
└─ importa Persistencia  (cargarDatos, guardarDatos, ...)

Interfaz.hs
├─ importa Tipos          (tipos de datos)
├─ importa Logica          (filtros, análisis, simulación)
└─ importa Validaciones   (validarRegistro, validarFecha)

Logica.hs
└─ importa Tipos          (tipos de datos)

Validaciones.hs
└─ importa Tipos          (tipos de datos)

Persistencia.hs
└─ importa Tipos          (tipos de datos)

Tipos.hs
└─ (no importa módulos del proyecto)

```

Tipos.hs es el único módulo sin dependencias internas. Esto garantiza que **no haya ciclos de dependencia**.

## 4.4 Flujo de datos

**Al iniciar:**

```

Disco (CSV) → Persistencia.cargarDatos → [RegistroFinanciero]
                                         → [Presupuesto]
                                         → [Regla]
                                         ↓
                                         Main.iniciarInterfaz

```

**Durante la ejecución (ejemplo: agregar registro):**

```

Usuario digita datos
    ↓
Interfaz.leerDouble / leerFechaValidada / prompt
    ↓
Validaciones.validarRegistro → Either String RegistroFinanciero
    ↓ (Right)
Nueva lista: registros ++ [nuevoRegistro]
    ↓
Interfaz.iniciarInterfaz (llamada recursiva con lista actualizada)

```

**Al salir:**

```

[RegistroFinanciero] → Persistencia.guardarDatos → Disco (finanzas.csv)

```

```
[Presupuesto]      → Persistencia.guardarPresupuestos → Disco  
(presupuestos.csv)  
[Regla]           → Persistencia.guardarReglas → Disco (reglas.csv)
```

El estado del sistema nunca se almacena en variables mutables globales. Las listas de datos se pasan explícitamente como argumentos siguiendo el **paradigma funcional puro**. La única mutación de estado ocurre en I/O: lectura de teclado y escritura/lectura de archivos.